FINANCIAL ENGINEERING

KBP FFÚ & KSTP FIS

Case Studies in Capital Markets and Banking

Lecture 1

Introduction to R

Ing. Milan Ficura, Ph.D.
milan.ficura@vse.cz

EVROPSKÁ UNIE
Evropské strukturální a investiční fondy
Operační program Výzkum, vývoj a vzdělávání

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

Lecture 1:
Introduction to R

Introduction to R

- **Computational software**
- Installation of R and RStudio
- RStudio layout
- Data types and basic operations
- Vectors and sequences
- Conditions and cycles
- User build functions
- Matrices and matrix operations
- Lists and data frames
- Loading data into R
- Charts and figures

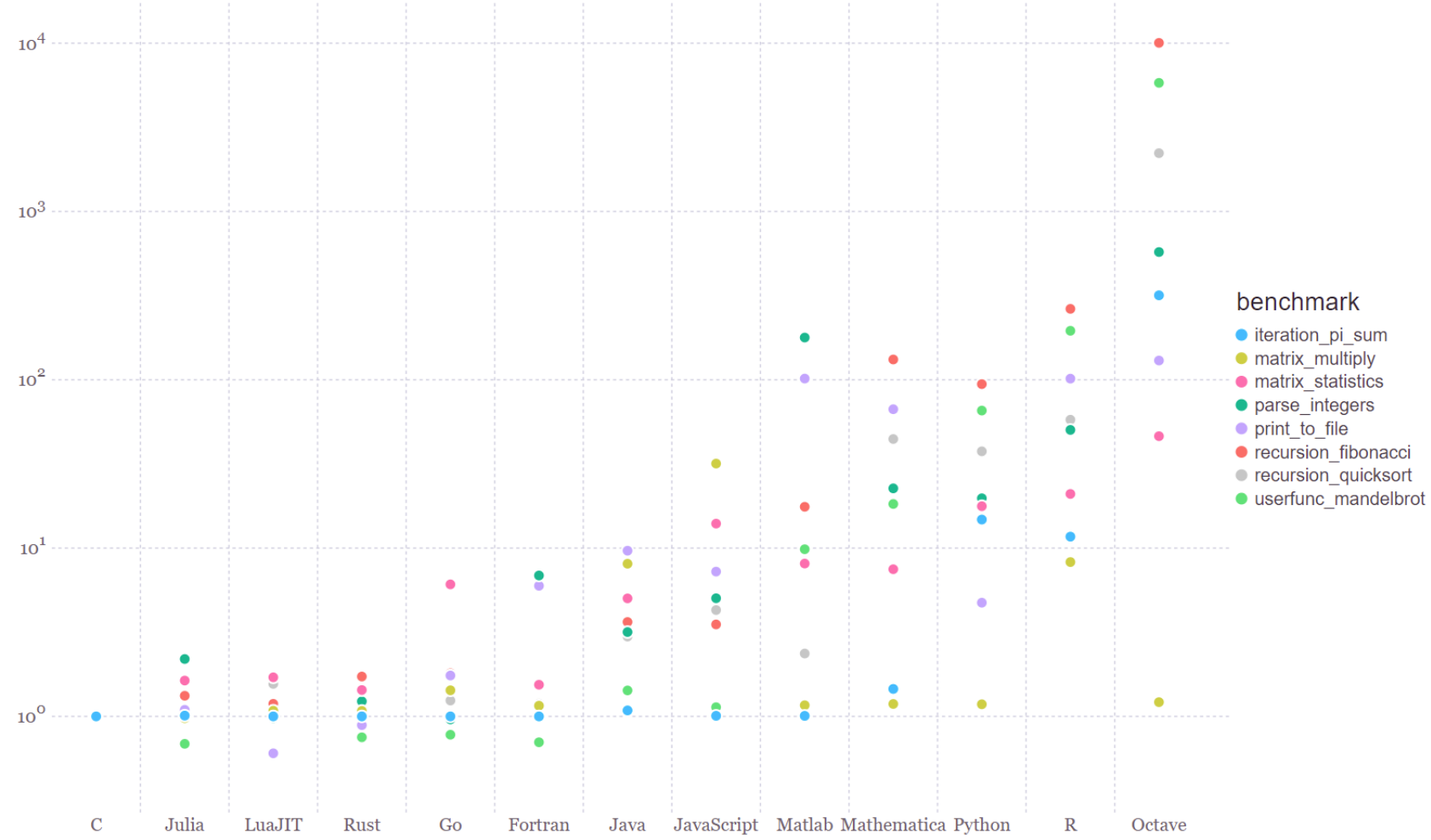
Main analytical tools used in finance

Software/language	Description	Cost
MATLAB®	Universal analytical software and programming language (+) Easy to use professionally build toolboxes (+) Fastest among the 4 main analytical programms (-) More narrow statistical functionalities than in R (-) Expensive (especially in the commercial version)	Home: 119€ + 35€ per toolbox Commercial: 2000€ + 1000€ (cca) per toolbox
RStudio®	IDE for the R language for statistical computing (+) Broadest statistical functionalities out of the 4 programs (+) Standard version is free (-) Need to learn large number of libraries (-) Many functions are not well optimized for speed	Free
python	Open-source general purpose programming language (+) Broadest machine learning functionalities (+) Free general purpose programming language (-) Less statistical functionalities than R (-) Generally slower than Matlab (but faster than R)	Free
sas	Professional analytical software and programming language (+) Easy to use professionally build functions (+) Support of SQL and databases connectivity (-) More narrow statistical/machine learning functionalities (-) Very expensive	8700$ per year (basic version)

Additional analytical tools

Software/language	Description	Cost
GNU Octave	Open-source alternative to Matlab (+) Provides most of the basic Matlab functionalities for free (-) Functionalities from toolboxes are often missing (-) Much slower than Matlab (or even R and Python)	Free
julia	Open-source high-performance programming language (+) Optimized for analytical computing (faster than Python) (-) More narrow libraries than for Python	Free
STATA	General purpose statistical software (+) Professionally build functions (easier to use than R) (-) Less flexible than R	765$ per year (basic version)
SPSS®	General purpose statistical software (+) Professionally build functions (easier to use than R) (-) Less flexible than R	99$ per month
EViews®	Econometric software (+) Professionally build functions (easier to use than R) (-) Less flexible than R	1695$ (standard licence)
Excel	General purpose data analysis program (+) Simple to use and to build applications with VBA (-) Missing advanced statistical functionalities (-) Very slow for large-scale computations	3650 Kč (for MS Office Pro)

Performance compared to C



Source: https://julialang.org/benchmarks/

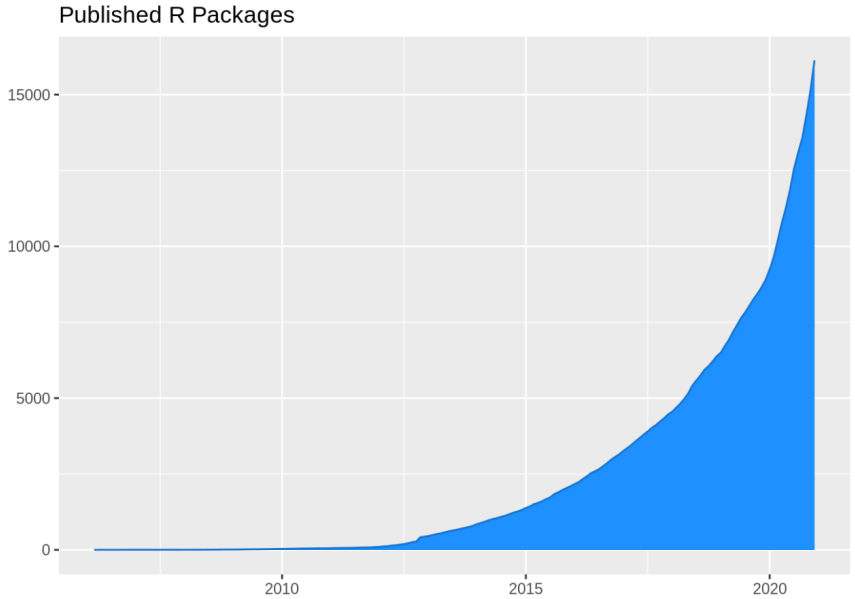
Introduction to R

- ✓ Computational software
- ➢ **Installation of R and RStudio**
- ❖ RStudio layout
- ❖ Data types and basic operations
- ❖ Vectors and sequences
- ❖ Conditions and cycles
- ❖ User build functions
- ❖ Matrices and matrix operations
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

Introduction to R & RStudio

- **R**
  - Open-source programming language for statistical computing
  - Available on: https://cran.r-project.org/
- **RStudio**
  - Integrated development environment (IDE) for the R programming language
  - Available on: https://rstudio.com/
- **R packages**
  - User-build extensions that extend the functionalities of R
  - There are currently over 15 000 R packages in the CRAN database (May 2020)

R packages in the CRAN database (May 2020):



Source: https://www.r-bloggers.com/2020/05/use-existing-functions-and-data-through-packages/

Instalation of R

1. Go to **CRAN R-Project webpage**: cran.r-project.org
2. Click on **Download R** (Linux, Mac or Windows based on your OS):



CRAN
Mirrors
What's new?
Task Views
Search

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- Download R for Linux
- Download R for (Mac) OS X
- Download R for Windows

3. Click on **Install R for the first time:**

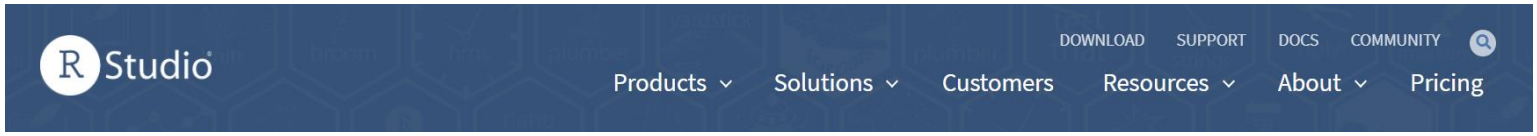
base — Binaries for base distribution. This is what you want to **install R for the first time**.

4. Download the **installer**, run it, and follow the instructions to install R:

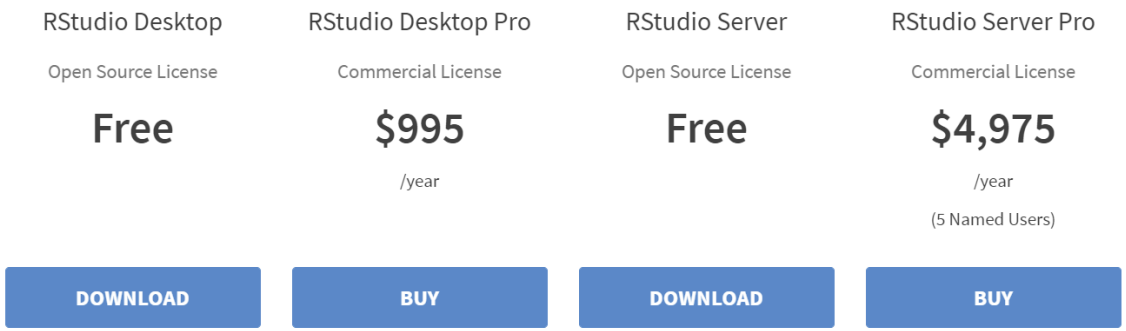
Download R 4.0.3 for Windows (85 megabytes, 32/64 bit)
Installation and other instructions
New features in this version

Instalation of RStudio

1. Go to **RStudio webpage**: rstudio.com
2. Click on **Download** (small button in the upper most pane):



3. Click on **Download** to download the **Free** version of **Rstudio Desktop:**



4. Download the **installer** corresponding to your operating system:

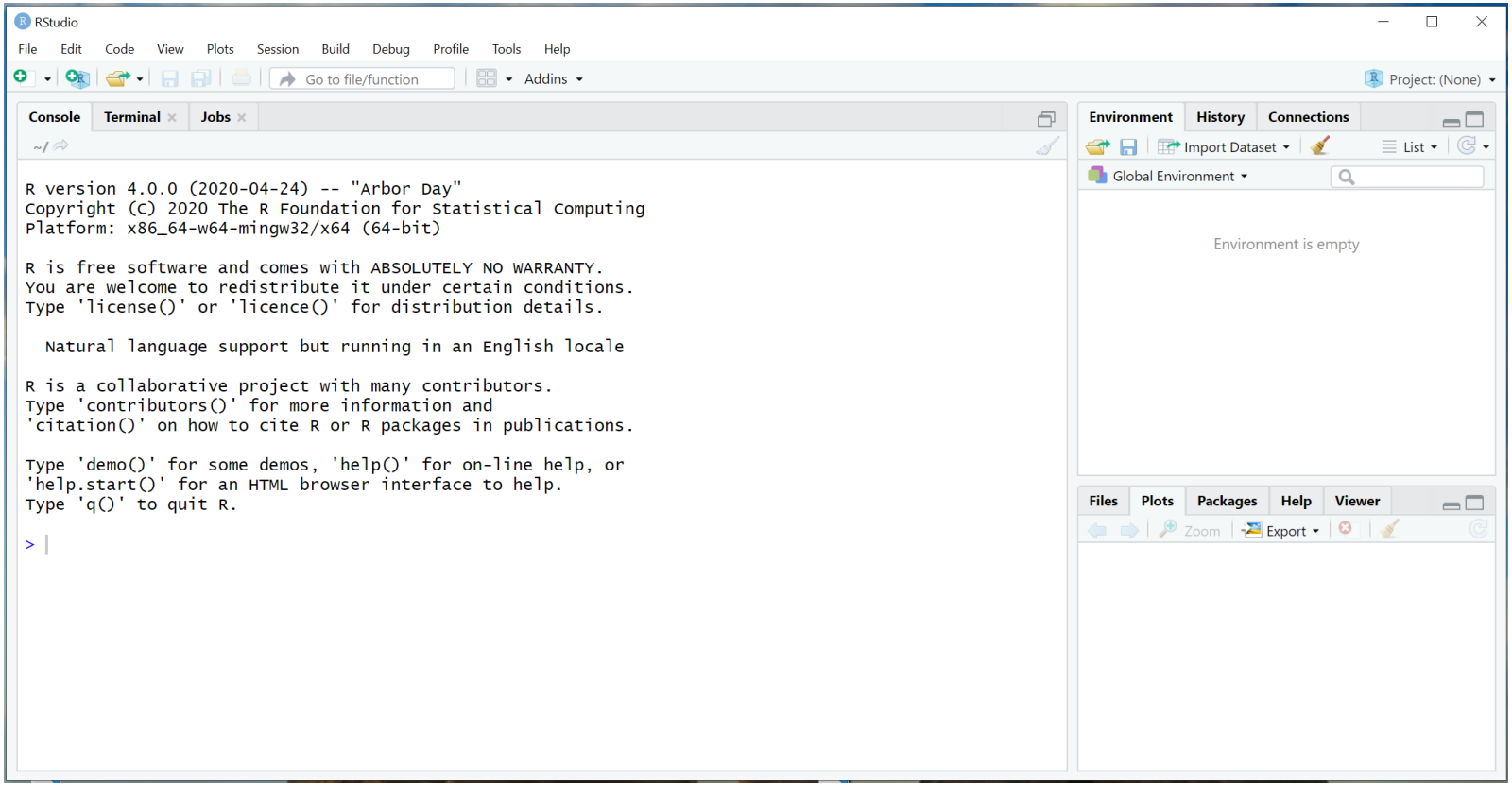
OS	Download	Size	SHA-256
Windows 10/8/7	RStudio-1.4.1103.exe	156.96 MB	c3384189
macOS 10.13+	RStudio-1.4.1103.dmg	152.77 MB	20148bd6
Ubuntu 16	rstudio-1.4.1103-amd64.deb	119.26 MB	f0857e27

5. Run the installer and follow the instructions to install RStudio:

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ➢ **RStudio layout**
- ❖ Data types and basic operations
- ❖ Vectors and sequences
- ❖ Conditions and cycles
- ❖ User build functions
- ❖ Matrices and matrix operations
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

RStudio interface



Left window:

- **Console →** Command line where you can type commands and see the results of executed commands
- **Terminal →** Access to system shell
- **Jobs →** Currently running scripts
- **Broom icon →** Clear the previously written commands

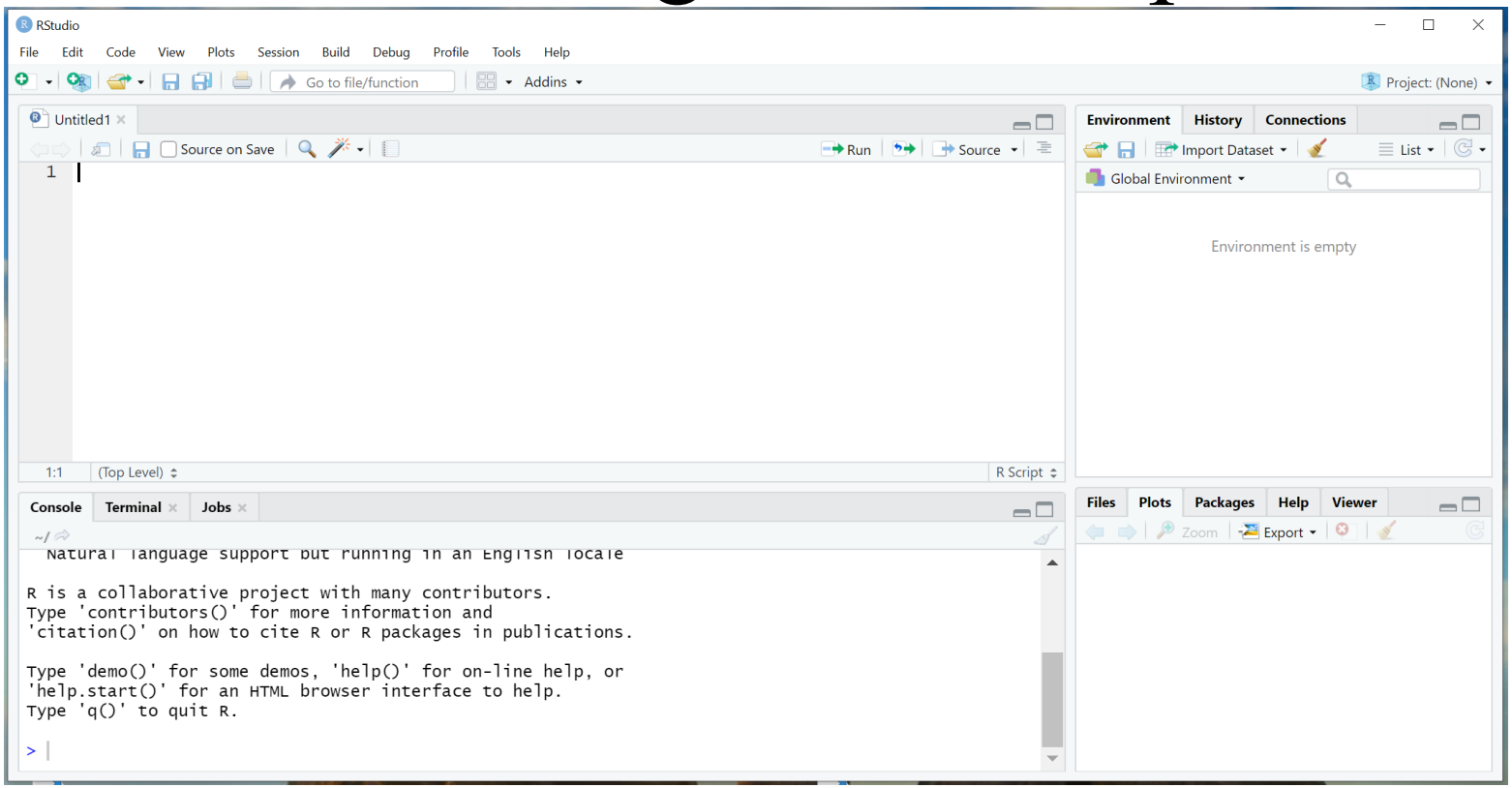
Upper-Right window

- **Enviroment →** Displays variables and data objects that you created
- **History →** History of commands that you have run in the past
- **Connections →** Data connections
- **Save icon →** Save the variables
- **Broom icon →** Clear the workspace

Lower-Right window

- **Files →** Current working directory
- **Plots →** Figures that you created in RStudio are displayed here
- **Packages →** Installed packages
- **Help →** Help to the RStudio
- **Viewer →** View local web content

Creating a new script

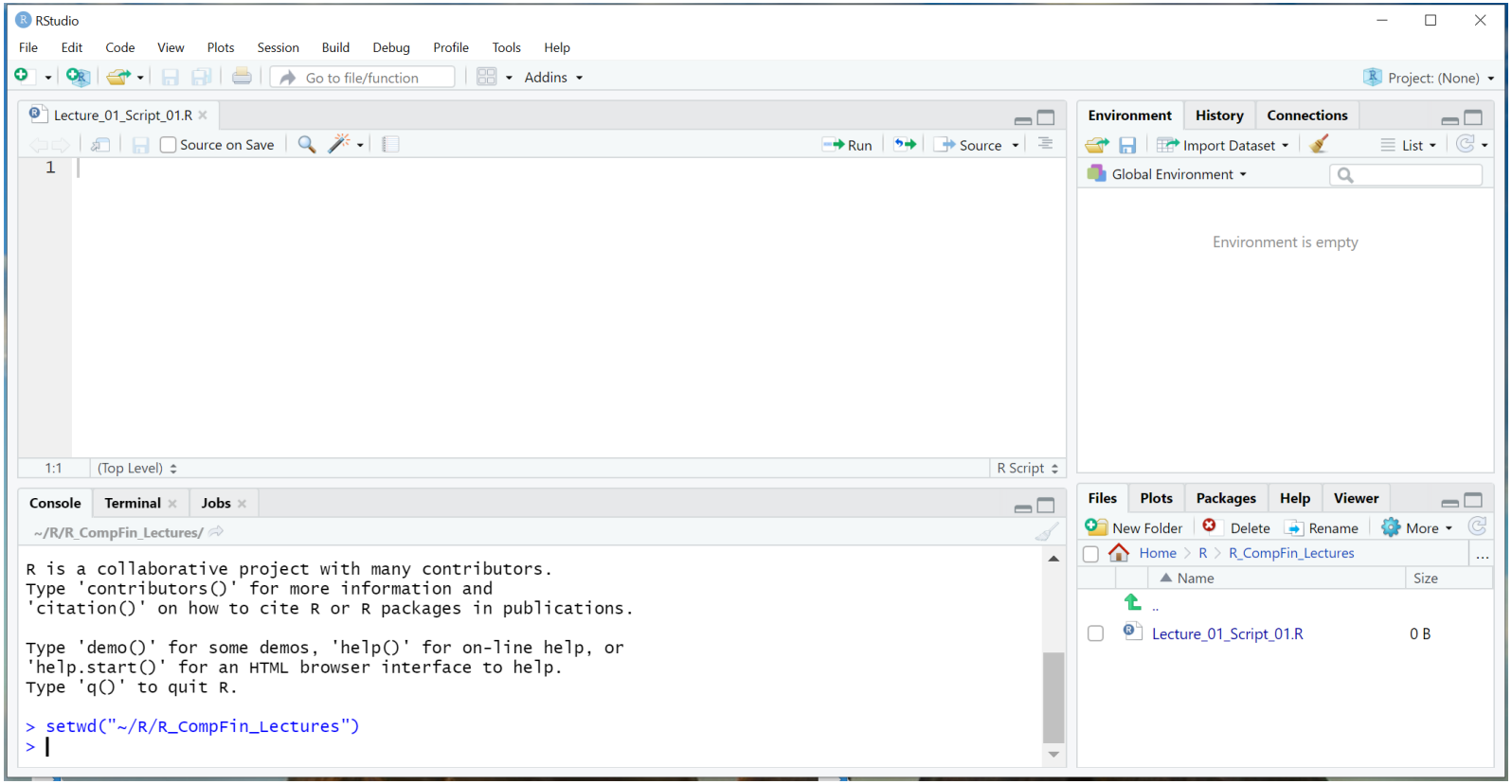


- **Scripts** allow you to save the code that you have written and run it again later
- To create a **new script** click on:
  - **File → New File → R Script**
- **Editor window** (with tab **Untitled1**) will appear, where you can write your new script

Editor window contains (among others):

- **Undock icon →** Allowing you to display multiple scripts at once
- **Save icon →** Save the current script
- **Search icon →** Search/replace parts of the script
- **Tools icon →** Some additional editing tools
- **Run icon →** Run the current line of code (or the selected part of the script)

Saving a new script



We will first **create a new folder** for this course and set it as the **working directory**:

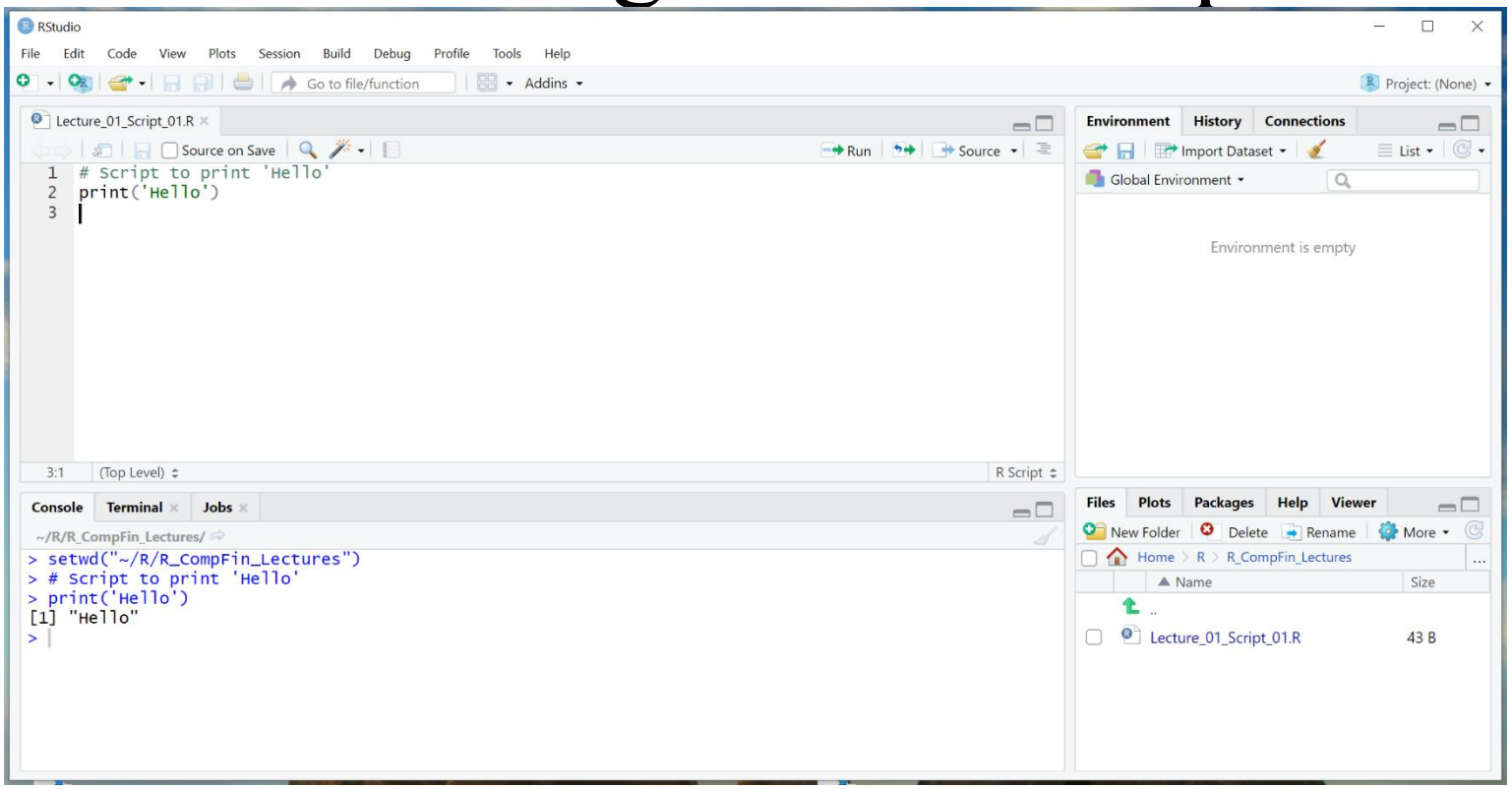
- In the **lower-right window**, click on the **Files tab**
- Find the **R folder**, open it, click on the **New Folder icon** and name the folder **R_CompFin_Lectures**
- Click on the **More icon** and select **Set as working directory** (this will allow us to run scripts directly from this folder without providing the path)

In order to **save the new script**:

- Click on **File → Save as** (or the Save icon)
- Save the new script to the newly created folder

The script will then appear in the working directory in the **Files window**, from where you can access it in the future (after you close it)

Writing the first script



We will write a script to print „Hello“ to the command window:

- For clarity of the code, it is good to **comment what it will do**, you can put comments to the code with a hashtag #, which can be written with ctrl+36
- On the next row, write: `print('Hello')`
- To run the code, put the cursor at the start of row 2 (or select the whole code) and click on the **Run icon**
- The result will appear in the command window

You can also try to:

- Save the finished script with the **Save icon**
- Close the script (with the x in its tab) and then open it again from the **Files widow**
- You can also run the script from the command line (or another R script) with the **source** command:

```
source('Lecture_01_Script_01.R')
```

(as long as the script is in the working directory otherwise you need to provide full path to it)

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ✓ RStudio layout
- ➢ **Data types and basic operations**
- ❖ Vectors and sequences
- ❖ Conditions and cycles
- ❖ User build functions
- ❖ Matrices and matrix operations
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

Basic features of R

- **Basic data types:**
  - **Numeric** → Real numbers with double precision (**basic R numeric data type**)
  - **Integer** → Integer numbers (used only if we want to save memory)
  - **Complex** → Complex numbers
  - **Logical** → Logical values (TRUE or FALSE)
  - **Character** → Characters and strings
  - **Raw** → Raw format (representing values as raw bytes)
- **Basic data structures:**
  - **Vector** → Basic data structure in R (single number = of size 1) (note: can also be non-numeric)
  - **Matrix** → Matrix composed of multiple vectors in its columns/rows
  - **Factor** → Data structure where inputs attain only limited number of values
  - **List** → Most general data type which can contain different variables/structures (including lists)
  - **Data frame** → Table that can contain different data types in its columns
- **Operators:**
  - **Mathematical operators** → Addition (+), Subtraction (-), Multiplica (*), Division (/), etc.
  - **Logical operators** → Larger than (>), Smaller than (<), Equal to (==), AND (&), OR(|), etc.
  - **Special operators in R** → Such as %IN% meaning „is part of“
- **Functions:**
  - **Mathematical functions** → Exponent (exp), Logarithm (log), Sine (sin), etc.
  - **Statistical functions** → Mean (mean), Standard deviation (sd), Quantiles (quantile), etc.
  - **Functions for data manipulation and many others**

Variable assignment

- **Variable assignment:**
  - **Leftwards assignment:** `x <- 5` (assigns value 5 to variable x)
  - **Rightwards assignment:** `5 -> x` (assigns value 5 to variable x)
  - **Equality assignment:** `x = 5` (assigns value 5 to variable x)

  (which can be used interchangeably in most cases)
- **Variable assignment to the parent environment:**
  - **Leftwards parent assignment:** `x <<- 5` (assigns 5 to x in the parent envir.)
  - **Rightwards parent assignment:** `5 ->> x` (assigns 5 to x in the parent envir.)

  (can be used to assign value from a function to the global environment)
- **Basic data types:**
  - **Numeric:** `x <- 5` or `x <- 5.555` or `x <- -5.555`
  - **Integer:** `x <- 5L`
  - **Complex:** `x <- 5 + 2i`
  - **Logical:** `x <- TRUE` or `X <- FALSE` or `X <- T` or `X <- F`
  - **Character:** `x <- 'Hello'` or `w <- "Hello"`
  - **Raw:** `x <- charToRaw('Hello')`
- **Checking for a variable type:**
  - **Class:** `class(x)` (high-level object type)
  - **Type:** `typeof(x)` (low-level object type)
  - **Length:** `length(x)` (length of the object)
  - **Attributes:** `attributes(x)` (meta-data of the object)

Basic mathematical functions (1)

- **Arithmetic operators:**
  - **Addition:** `z <- x + y`
  - **Subtraction:** `z <- x - y`
  - **Multiplication:** `z <- x * y`
  - **Division:** `z <- x / y`
  - **Exponentiation:** `z <- x ^ y` or `z <- x ** y`
  - **Root:** `z <- x ^ (1/y)` or `z <- x ** (1/y)`
  - **Modulus:** `z <- x %% y`
  - **Integer division:** `z <- x %/% y`

- **Basic mathematical functions:**
  - **Absolute value:** `z <- abs(x)`
  - **Square root:** `z <- sqrt(x)`
  - **Natural exponent:** `z <- exp(x)`
  - **Natural logarithm:** `z <- log(x)`
  - **Decadic logarithm:** `z <- log10(x)`
  - **Logarithm with base y:** `z <- log(x, base=y)`
  - **Factorial:** `z <- factorial(x)`
  - **Num. of combinations:** `z <- choose(x,y)` (when drawing y elements from x)
  - **Sign:** `z <- sign(x)`
  - **Rounding:** `z <- round(x)` or `z <- round(x,y)` (rounding to y digits)
  - **Floor and Ceiling:** `z <- floor(x)` and `z <- ceiling(x)`

Note: The inputs **x** and **y** into most of the operators and functions can be numbers (i.e. unit vectors), vectors or matrices. In the case when they are vectors or matrics, the operation will be performer in an element-wise way.

Basic mathematical functions (2)

Trigonometric functions:

- **Sine:** `z <- sin(x)` or `z <- sinpi(x)` (where x is in x*pi)
- **Cosine:** `z <- cos(x)` or `z <- cospi(x)` (where x is in x*pi)
- **Tangens:** `z <- tan(x)` or `z <- tanpi(x)` (where x is in x*pi)
- **Arcus sine:** `z <- asin(x)`
- **Arcus cosine:** `z <- acos(x)`
- **Arcus tangens:** `z <- atan(x)`
- **Hyperbolic sine:** `z <- sinh(x)`
- **Hyperbolic cosine:** `z <- cosh(x)`
- **Hyperbolic tangens:** `z <- tanh(x)`

Note: **cotangens** is not implemented in basic R, you can either compute it with `z <- cos(x)/sin(x)` or `z <- 1/tan(x)`, or install the package **pracma**, which adds functions **cot**, **acot**, **sec**, **asec**, etc.

Some other pre-build mathematical notation:

- **Quick exponentiation:** `z <- 2e10` (works for **numbers**: 2e4, 5e3, but **not variables**: xey)
- **Constant pi:** `z <- sin(pi/2)` (for **constant e** use `z <- exp(1)`)
- **+-Infinity:** `z <- Inf` and `z <- -Inf`
- **Not a number:** `z <- NaN` (the result of `z <- 0/0` or `z <- Inf/Inf`)
- **Not available:** `z <- NA` (general notation of a missing or erronous value)

Checking for specific mathematical values:

- **Complex numbers:** `is.complex(x)`
- **Infinite values:** `is.infinite(x)` (gives TRUE value for +Inf as well as –Inf)
- **NaN:** `is.nan(x)`
- **NA:** `is.na(x)` (gives TRUE for NA but also for NaN values)

Basic logical operators

- **Relational operators:**
  - **Less than:** `x < y`
  - **Greater than:** `x > y`
  - **Less or equal to:** `x <= y`
  - **Greater or equal to:** `x >= y`
  - **Equal to:** `x == y`
  - **Not equal to:** `x != y`
- **Logical operators and functions:**
  - **Logical NOT:** `!x`
  - **Logical AND:** `x & y`
  - **Logical OR:** `x | y`
  - **Non-vectorized AND:** `x && y` (x and y must not be vectors)
  - **Non-vectorized OR:** `x || y` (x and y must not be vectors)
  - **Logical XOR:** `xor(x,y)` (gives TRUE if x or y is true, but not both)
  - **Logical ALL:** `all(x,y,z)` (all conditions x,y,z must be TRUE to give TRUE)
  - **Logical ANY:** `any(x,y,z)` (if any of the conditions is TRUE, it gives TRUE)
  - **Is TRUE:** `isTRUE(x)` (non-vectorized, usefull for handling the NA case)
  - **Is FALSE:** `isTRUE(y)` (non-vectorized, usefull for handling the NA case)

Note: Apart from the few non-vectorized operations, the variables **x** and **y** can be vectors and matrices, and the mathermatical/logical operations will be applied element-wise

Other usefull functions

- **Other usefull functions:**
  - **Print variable z:** `print(z)`
  - **Create empty variable:** `z <- NULL` (usefull for recursivelly appending it)
  - **Delete variable x:** `rm(x)`
  - **Clear workspace:** `rm(list=ls())`
  - **Check if x exists:** `exists("x")`
  - **System time:** `Sys.time()` (usefull for measuring time of code execution)
  - **System date:** `Sys.Date()`
  - **Call help for NAME:** `?NAME`
  - **Set working directory:** `setws("PATH")`
  - **Run a script:** `source("NAME")` (must be in the working directory)
  - **Install R package:** `install.packages("NAME")`
  - **Load R package:** `library(NAME)`
  - **List of loaded packages:** `sessionInfo()`

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ✓ RStudio layout
- ✓ Data types and basic operations
- ➢ **Vectors and sequences**
- ❖ Conditions and cycles
- ❖ User build functions
- ❖ Matrices and matrix operations
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

Vectors - Introduction

- **Vectors:**
  - Basic data structure in R → Even a single number is viewed as a vector of size 1
  - Can contain any of the basic data types (number, integer, logical, string, etc.)
  - But each vektor can be composed only from variables of a single type
- **Create empty vectors:**
  - **Create numeric vector of size k:** `z <- numeric(k)`
  - **Create logical vector of size k:** `z <- logical(k)`
  - **Create empty string size k:** `z <- character(k)`
- **Example:**

```
# Create numeric vector (of zeros) of size 6
numeric(6)
[1] 0 0 0 0 0 0

# Create logical vector (of FALSE) of size 6
logical(6)
[1] FALSE FALSE FALSE FALSE FALSE FALSE

# Create empty character vector of size 6
character(6)
[1] "" "" "" "" "" ""
```

Vectors and sequences

- **Create a custom vector:**
  - **Create custom numeric vector:** `z <- c(1,2,-2,5,3.5,pi)`
  - **Create custom logical vector:** `z <- c(T,T,F,T,F,F)`
  - **Create custom string vector:** `z <- c('A','B','C','D','E','F')`
- **Create a vector as sequence:**
  - **Sequences of numbers from x to y:** `z <- x:y` or `z <- seq(x,y)`
  - **Sequence from x to y with step s:** `z <- seq(x,y,s)`
  - **Vector of x repeated n times:** `z <- rep(x,n)`
- **Vector referencing:** (Note: x will now be a vector)
  - **Select i-th value from a vector:** `z <- x[i]`
  - **Select i-th to j-th value:** `z <- x[i:j]`
  - **Select i-th, j-th and k-th value:** `z <- x[c(i,j,k)]`
  - **Select all values except i-th one:** `z <- x[-i]`
  - **Select based on logical values:** `z <- x[c(T,T,F,T,F,F)]`
- **Mathematical operations:** (Note: x and y will now be vectors)
  - **Element-wise addition:** `z <- x + y` (z will be a vector)
  - **Element-wise product:** `z <- x * y` (z will be a vector)
  - **Element-wise division:** `z <- x / y` (z will be a vector)
  - **Inner product:** `z <- x %*% y` (z will be a scalar)
  - **Outer product:** `z <- x %o% y` (z will be a matrix)

(Note: Most of the previously mentioned mathematical functions work in an element-wise way)

Vectors functions

❖ Basic vector functions:

- **Concatenate two vectors:** `z <- c(x,y)`
- **Repeat vector n times:** `z <- rep(x,n)` or `z <- rep(x,times=n)`
- **Repeat each element n times:** `z <- rep(x,each=n)`
- **Length of a vector:** `z <- length(x)`
- **Sum of vector elements:** `z <- sum(x)`
- **Product of vector elements:** `z <- prod(x)`
- **Maximum or minimum of a vector:** `z <- max(x)` or `z <- min(x)`
- **Sort the vector (ascending):** `z <- sort(x)`
- **Sort the vector (descending):** `z <- sort(x,decreasing=TRUE)`
- **Rank of the vector elements:** `z <- rank(x)`
- **Order indices of the elements:** `z <- order(x)`
- **Unique elements in a vector:** `z <- unique(x)`
- **Return first or last n elements:** `z <- head(x,n)` or `z <- tail(x,n)`
- **Apply function to each element:** `z <- sapply(x,FUN)`
- **Plot vector values on a chart:** `plot(x)`

❖ Logical functions:

- **Check if vector y contains x:** `z <- x %in% y` (output is logical vector of size x)
- **Indices of TRUE elements:** `z <- which(x)` (x must be a logical vector)
- **Logical ALL and ANY:** `z <- all(x)` (for T, all conditions in x must be T)
- **Logical ANY:** `z <- any(x)` (for T, any condition in x must be T)

Statistical functions

- **Summary statistics:**
  - **Mean:** `z <- mean(x)`
  - **Median:** `z <- median(x)`
  - **Variance:** `z <- var(x)`
  - **Standard deviation:** `z <- sd(x)`
  - **Standard z scores:** `z <- scale(x)`
  - **Quantile:** `z <- quantile(x)`
- **Statistical analysis:** (Note: only a small selection)
  - **t-test that mean(x)=y:** `t.test(x,mu=n)`
  - **Correlation:** `cor(x,y)`
  - **Correlation test:** `cor.test(x,y)`
  - **Univariate linear regression:** `lm(y~x)` with constant, otherwise: `lm(y~0+x)`
  - **Multivariate linear regression:** `lm(y~x1+x2+x3)`
- **Statistical distributions:** (Note: we mention only the basic ones)
  - **Normal density (pdf):** `dnorm(x, mean, sd)`
  - **Normal cum. density (cdf):** `pnorm(x, mean, sd)`
  - **Normal quantile function:** `qnorm(p, mean, sd)`
- **Random numbers:** (Note: we mention only the basic ones)
  - **Gaussian random numbers:** `rnorm(n, mean, sd)`
  - **Uniform random numbers:** `runif(n)`
  - **Exponential random numbers:** `rexp(n,p)`

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ✓ RStudio layout
- ✓ Data types and basic operations
- ✓ Vectors and sequences
- ➢ **Conditions and cycles**
- ❖ User build functions
- ❖ Matrices and matrix operations
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

Conditions and loops

- **Conditional statements**
  - **ifelse function** `z <- ifelse(CONDITION,x,y)`
  - **if statement** `if(CONDITION){ACTION}`
  - **if else statement** `if(CONDITION){ACTION}ifelse(CONDITION2){ACTION}`
  - **else statement** `if(CONDITION){ACTION}else{ACTION}`
- **Loop statements:**
  - **for statement** `for (i in 1:k){LOOP}`
  - **break statement** `if(CONDITION){break}`
  - **next statement** `if(CONDITION){next}`
  - **while statement** `while(CONDITION){LOOP}`
  - **repeat statement** `repeat{LOOP if(CONDITION){break}}`
- **Note:**
  - Apart from the **ifelse** function, the codes above are only schematic, as R requires the other loops to be writen over multiple rows, with exact placement of the {} brackets
  - The logic of the different types of conditional statements will best be explained on concrete examples

Example: ifelse function

- **Task:** Create an ifelse script to evaluate whether number x is even or odd

Example: ifelse function

- **Task:** Create an ifelse script to evaluate whether number x is even or odd

- **Solution:**

```
# Script to evaluate whether number x is even or odd
# By using an ifelse statement and a modulo (%%) operator

# Input your number here:
x <- 6

# Calculate result:
y <- ifelse(x%%2==0,'Even Number','Odd Number‘)

# Print result:
print(y)
```

- **Output from R:**

```
print(y)
[1] "Even Number"
```

Example: if-else if-else statement

- **Task:** Write an if-else if-else statement to check if a number is positive, negative or zero

Example: if-else if-else statement

- **Task:** Write an if-else if-else statement to check if a number is positive, negative or zero
- **Solution:**

```
# EXAMPLE 2: Script to evaluate whether number x is positive,
# negative or zero by using an if-else if-else statement
# NOTE: Input must be a number and not NaN or NA
# Input your number here:
x <- 6
# If-else if-else statement:
if (x > 0){
  print ('Positive Number')
} else if (x < 0){
  print ('Negative Number')
} else {
  print ('Zero Number')
}
```

- **Output from R:**

```
[1] "Positive Number"
```

Example: for loop

- **Task:** Use a for loop to compute the k-th factorial

Example: for loop

- **Task:** Use a for loop to compute the k-th factorial
- **Solution:**

```
# For loop to compute the k-th factorial
# Order of the factorial
k <- 6
# For loop to compute the factorial
x <- 1
for (i in 1:k) {
  x <- x*i
}
# Print the result
print(x)
```

- **Output from R:**

```
[1] 720
```

- **Check of the result:**

```
factorial(k)
[1] 720
```

- **Check of the result (alternative):**

```
sum(prod(1:k))
[1] 720
```

Example: for loop

- **Task:** Assume that the stock of a company follows the random walk process $S_t = S_{t-1} + \varepsilon_t$ where $\varepsilon_t \sim N(\mu, \sigma)$. Use a for loop to simulate it and draw its evolution with the plot function

Example: for loop

- **Task:** Assume that the stock of a company follows the random walk process $S_t = S_{t-1} + \varepsilon_t$ where $\varepsilon_t \sim N(\mu, \sigma)$. Use a for loop to simulate it and draw its evolution with the plot function
- **Solution:**

```
# For loop to simulate the random walk process of a stock price
# Parameters
S0 <- 100  # Current stock price
mju <- 0.1 # Drift rate per day
sigma <- 3 # Volatility per day
T <- 252   # Number of days of the simulation
# Empty vector to which we will write the stock price evolution
S <- rep(NaN,t+1) # Take note that S[1] corresponds to t=0
S[1] <- S0
# For loop to simulate the stock price evolution
for (i in 2:(t+1)){
  # Simulation
  S[i] <-  S[i-1] + rnorm(1,mju,sigma)
}
# Draw the stock price evolution
plot(S)
title('Stock price - Arithmetic Brownian Motion')
```

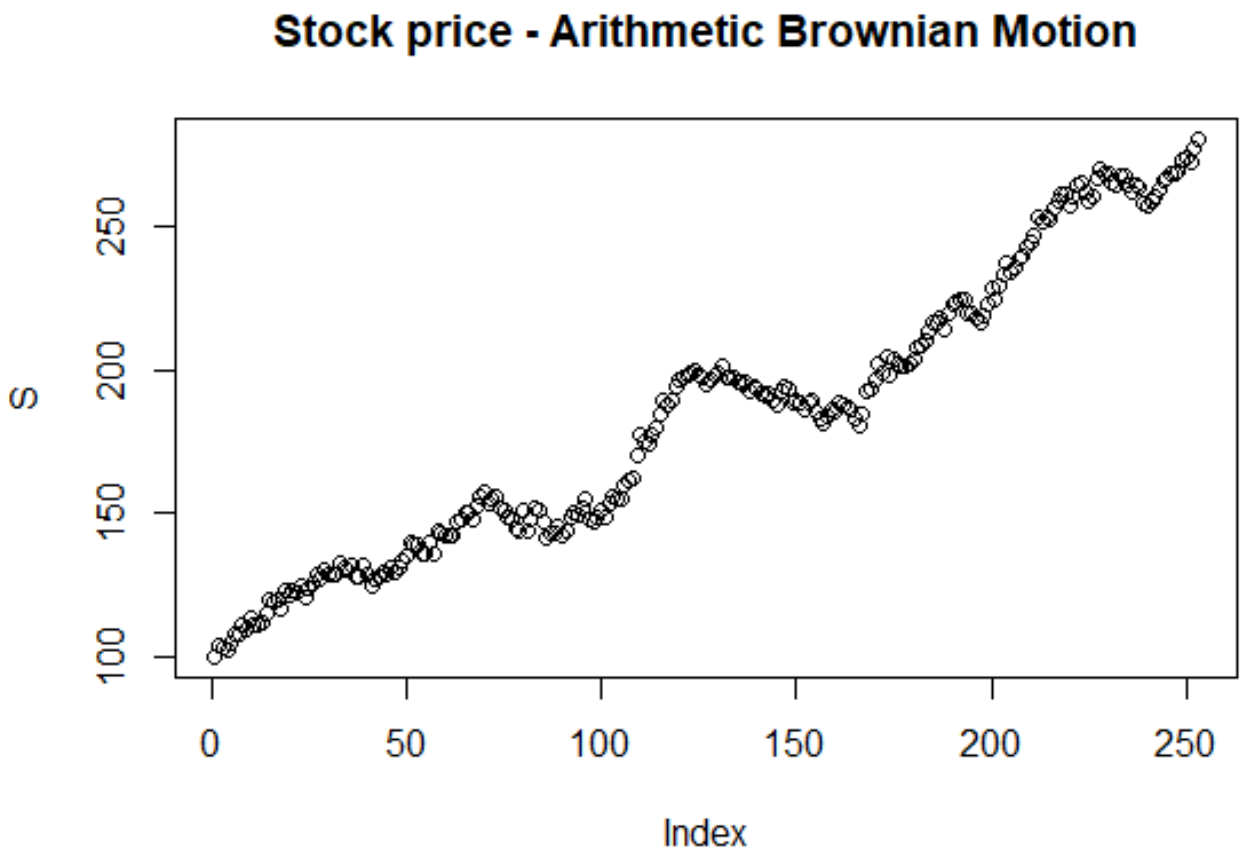
Alternative

- **Task:** Assume that the stock of a company follows the random walk process $S_t = S_{t-1} + \varepsilon_t$ where $\varepsilon_t \sim N(\mu, \sigma)$. Use the cumsum function to simulate it and draw its evolution with the plot function
- **Solution:**

```
# Simulate the random walk process using cumsum function
S <- c(S0,S0+cumsum(rnorm(t,mju,sigma)))
# Draw the stock price evolution
plot(S)
title('Stock price - Arithmetic Brownian Motion‘)
```

- **Note:** For simpler independent processes it is usually easier to use the cumsum function to perform the simulation. If we wanted, for example, to simulate the AR process, we would need to use the FOR loop as the process contains dependency between observations in time

Random walk simulation plot



Note: The random walk of S(t) corresponds to the discretized Arithmetic Brownian Motion process, which implies linear growth of the stock price (instead of exponential). We will look at the simulation of more advanced processes of the stock price (GBM, Heston, SABR, etc.) in the later parts of this course.

Example: for-break loop

- **Task:** Unlike more realistic price processes (such as Geometric Brownian Motion), the random walk process (Arithmetic Brownian Motion) allows for the stock price to become negative. Extend the previous script to identify such an event, stop the simulation and print that the company defaulted.

Example: for-break loop

- **Task:** Unlike more realistic price processes (such as Geometric Brownian Motion), the random walk process (Arithmetic Brownian Motion) allows for the stock price to become negative. Extend the previous script to identify such an event, stop the simulation and print that the company defaulted.
- **Modification of the loop in the previous script:**

```
# …
# For loop to simulate the stock price evolution
for (i in 2:(t+1)){
  # Simulation
  S[i] <-  S[i-1] + rnorm(1,mju,sigma)
  # Evaluation of the default event (THE NEW PART)
  if (S[i] <= 0){
    print(paste('Company defaulted on day', i-1))
    break
  }
}
# …
```

Example: while loop

- **Task:** Write a loop that continues drawing simulations of the stock price for an indefinite period until one of the following two events occurrs: (1) stock price drops to zero (i.e. stock defaults), or (2) stock reaches a profit target that is 10x larger than the current stock price

Example: while loop

- **Task:** Write a loop that continues drawing simulations of the stock price for an indefinite period until one of the following two events occurrs: (1) stock price drops to zero (i.e. stock defaults), or (2) stock reaches a profit target that is 10x larger than the current stock price
- **Solution:**

```
# Parameters
S0 <- 100   # Current stock price
mju <- 0.5   # Drift rate per day
sigma <- 3  # Volatility per day
PT <- 1000 # Profit target
# While loop to simulate the stock price evolution until default or
reaching of PT
S <- S0
Si <- S0
while ((Si>=0) & (Si<=PT)) {
  # Simulate the stock price on day i
  Si <-  Si + rnorm(1,mju,sigma)
  # Append Si to the price evolution
  S <- c(S,Si)
}
```

Example: repeat-break loop

- **Task:** Rewrite the previous script with a repeat-break loop and print a statement telling us which of the two possible outcomes occurred

Example: repeat-break loop

- **Task:** Rewrite the previous script with a repeat-break loop and print a statement telling us which of the two possible outcomes occurred
- **Solution:**

```
# Repeat-Break loop to simulate the stock price evolution until
default or reaching of PT
S <- S0
Si <- S0
i <- 0
repeat {
# Simulate the stock price on day i
  i <- i+1
  Si <-  Si + rnorm(1,mju,sigma)
  S <- c(S,Si)
  # Break the loop
  if (Si <= 0){
    print(paste('Company defaulted on day', i))
    break
  } else if (Si >= PT){
    print(paste('Company reached PT on day', i))
    break
  }
}
```

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ✓ RStudio layout
- ✓ Data types and basic operations
- ✓ Vectors and sequences
- ✓ Conditions and cycles
- ➢ **User build functions**
- ❖ Matrices and matrix operations
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

User build functions

- **Conditional statements**
  - **inline function:** `name <- function(x) OPERATIONS(x)`
  - **standard function:** `name <- function(x) {OPERATIONS}`
- **Note**
  - The main difference is that while **inline** function is written on a single row, the **standard function** can (must) be written over multiple rows (and can thus contain large number of operations, including conditions, cycles, etc.)
  - Both of the functions **can have more than a single input:** `function(x,y,z)`
- **Setting default options**
  - **default inputs:** `name <- function(x=1) {OPERATIONS}`
- **Note**
  - i.e. if the user does not specify argument x, a default value will be used instead
- **Getting output from a function**
  - **return statement:** `name <- function(x) {OPERATIONS return(z)}`
  - **parent assignment:** `name <- function(x) {OPERATIONS z<<-y}`
- **Note**
  - In R the return statement can produce **only a single output** `return(z)`
  - To produce multiple outputs, we can set the output **z** as a list (which we will do later)
  - Alternative way of how to make function produce outputs is with the **parent assignment**, which allows us to change variables in the global environment (i.e. replace global variable z with y)

Example: inline function

- **Task:** Write an inline function to compute the **logistic function:** $\Lambda(x) = \frac{1}{1+e^{-x}}$

Example: inline function

- **Task:** Write an inline function to compute the **logistic function**
- **Solution:**

```
# Inline function to compute the logistic function
# Define x
x <- 1
# Logit function
mylogfun <- function(x) 1/(1+exp(-x))
# Calculate result
mylogfun(x)
```

Logistic function:

$$\Lambda(x) = \frac{1}{1 + \mathrm{e}^{-x}}$$

- **Output from R:**

```
[1] 0.7310586
```

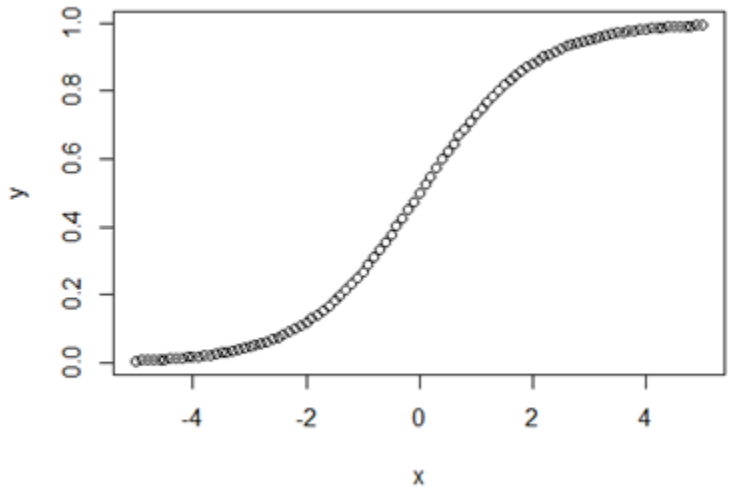
- **We can also apply it to a vector:**

```
# Vector input
x <- seq(-5,5,by=0.1)
y <- mylogfun(x)
```

- **And plot the result**

```
# Draw the chart of the function
plot(x,y)
```

Logistic function:



Example: standard function

- **Task:** Rewrite the **logistic function** as a standard user defined function instead

Example: standard function

- **Task:** Rewrite the **logistic function** as a standard user defined function instead
- **Solution:**

```
# Standard user function to compute the logistic function
# Define x
x <- 1
# Logit function
mylogfun2 = function(x) {
  z <- 1/(1+exp(-x))
  return(z)
}
# Calculate result
mylogfun2(x)
```

Logistic function:

$$\Lambda(x) = \frac{1}{1 + \mathrm{e}^{-x}}$$

- **Output from R:**

```
[1] 0.7310586
```

Example: standard function

- **Task:** Extend the previous logit function to a **general logistic function** with 3 parameters (x_0, k, L) in addition to the input x. Write the function in a way so that uses default values $x_0 = 0$, $k = 1$, $L = 1$ if no inputs are given.

Logistic function:

$$\Lambda(x) = \frac{1}{1 + \mathrm{e}^{-x}}$$

General logistic function:

$$\Lambda(x) = \frac{L}{1 + \mathrm{e}^{-k(x - x_0)}}$$

Parameters:

- x_0 = Sigmoid midpoint
- k = Curve steepness
- L = Maximum value

Example: standard function

- **Task:** Extend the previous logit function to a **general logistic function** with 3 parameters (x_0, k, L) in addition to the input x. Write the function in a way so that uses default values $x_0 = 0$, $k = 1$, $L = 1$ if no inputs are given.
- **Solution:**

```
# Define x
x <- 1
# Logistic function
mylogfun2 = function(x,x0=0,k=1,L=1) {
  z <- L/(1+exp(-k*(x-x0)))
  return(z)
}
```

- **Result – No additional parameters:**

```
mylogfun2(x)
[1] 0.7310586
```

- **Result – With additional parameters:**

```
# Set additional parameters
x0 <- 2 # Sigmoid midpoint
k <- 3 # Curve steepness
L <- 5 # Maximum value
# Calculate result
mylogfun2(x,x0,k,L)
[1] 0.237194
```

Logistic function:

$$\Lambda(x) = \frac{1}{1 + \mathrm{e}^{-x}}$$

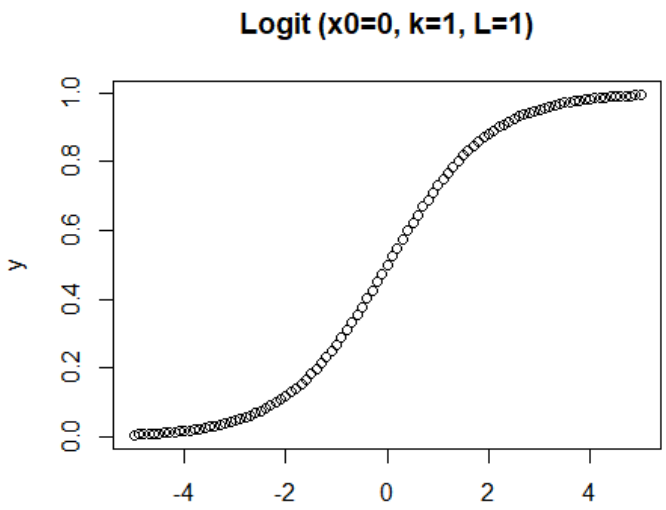
General logistic function:

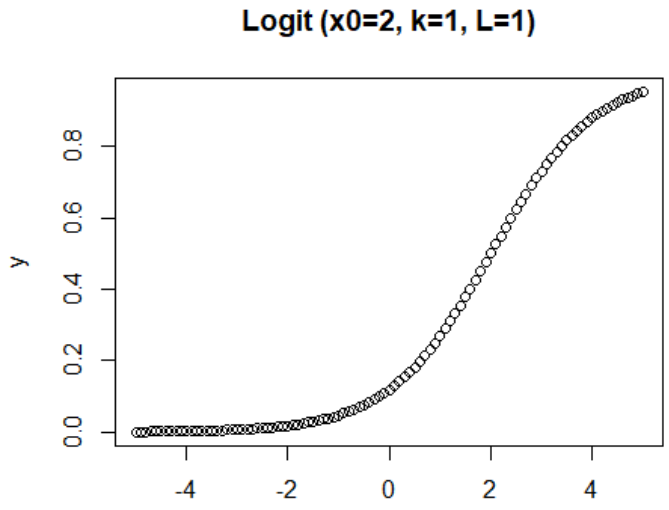
$$\Lambda(x) = \frac{L}{1 + \mathrm{e}^{-k(x-x_0)}}$$

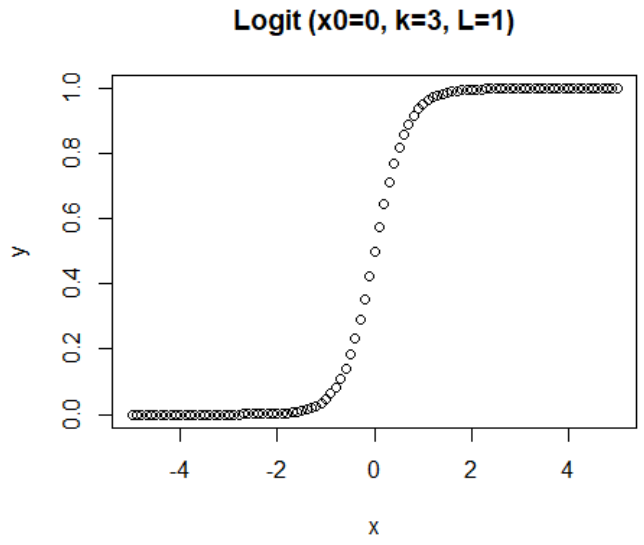
Parameters:

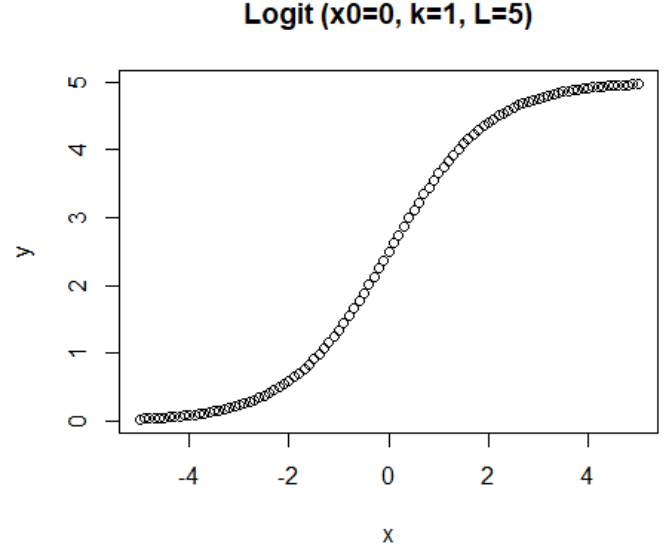
- x_0 = Sigmoid midpoint
- k = Curve steepness
- L = Maximum value

Comparison (for -5<x<5)









Example: user function with cycle

- **Task:** Write an user function to compute the Fibonacci sequence of order k

Fibonacci sequence is defiend as follows:

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$
- for $n > 1$

The first terms of the sequence are:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, …

Example: user function with cycle

- **Task:** Write an user function to compute the Fibonacci sequence of order k
- **Solution:**

```
# Function to compute Fibonacci sequence of order k
myfibonacci = function(k) {
  # Create an empty vector of size k+1
  # (as we start with the 0-th term)
  fibonacci = numeric(k+1)
  # Cycle to get the Fibonacci sequence
  for (i in 1:(k+1)) {
    if (i==1) {
      fibonacci[i] <- 0
    } else if (i==2) {
      fibonacci[i] <- 1
    } else {
      fibonacci[i] <- fibonacci[i-1] + fibonacci[i-2]
    }
  }
  # Return result
  return(fibonacci)
}
```

Fibonacci sequence is defiend as follows:

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$
- for $n > 1$

The first terms of the sequence are:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, …

- **Apply the function:**

```
k=15 # We set the order to 15
myfibonacci(k) # Run the function
[1] 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610
```

Example: parent assignment

- **Task:** Extend previous code to save the information about the computation time

Example: parent assignment

- **Task:** Extend previous code to save the information about the computation time
- **Solution:** (we will use the parent assignment feature: `<<-`)

```
# Function to compute Fibonacci sequence of order k
myfibonacci = function(k) {
  # Start of calculation time
  t1 <- Sys.time()
  # Create an empty vector of size k+1 (as we start with the 0-th term)
  fibonacci = numeric(k+1)
  # Cycle to get the Fibonacci sequence
  for (i in 1:(k+1)) {
    if (i==1) {
      fibonacci[i] <- 0
    } else if (i==2) {
      fibonacci[i] <- 1
    } else {
      fibonacci[i] <- fibonacci[i-1] + fibonacci[i-2]
    }
  }
  # End of calculation time
  t2 <- Sys.time()
  # Save the calculation time to the global enviroment
  t <<- t2-t1
  # Return result
  return(fibonacci)
}
```

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ✓ RStudio layout
- ✓ Data types and basic operations
- ✓ Vectors and sequences
- ✓ Conditions and cycles
- ✓ User build functions
- ➢ **Matrices and matrix operations**
- ❖ Lists and data frames
- ❖ Loading data into R
- ❖ Charts and figures

Matrices – Introduction

- **Matrices:**
  - Vectors stacked row-wise or column-wise into a matrix
  - Can contain any data type, but all elements must be of the same data type
  - Allow us to perform matrix operations

Matrices

- **Create a custom matrix:**
  - **Create an k*q NA matrix:** `z <- matrix(nrow=k,ncol=q)`
  - **Convert vector x into k*q matrix:** `z <- matrix(x,k,q)` (x can be a unit vector)
  - **Create diagonal matrix of size k:** `z <- diag(k)` (if k is vector it will be the diagonal)
  - **Bind vectors column-wise:** `z <- cbind(x,y)` (x and y are vectors)
  - **Bind vectors row-wise:** `z <- rbind(x,y)` (x and y are vectors)
- **Basic matrix functions:** (x and y are now matrices)
  - **Dimensions of a matrix:** `z <- dim(x)`
  - **Number of rows/columns:** `z <- nrow(x)` and `z <- ncol(x)`
  - **Number of elements:** `z <- length(x)`
  - **Extract the diagonal elements:** `z <- diag(x)`
  - **Bind matrices row/column wise:** `z <- cbind(x,y)` and `z <- rbind(x,y)`
  - **Convert matrix to vector:** `z <- as.vector(x)`
- **Matric referencing:** (Note: x will now be a vector)
  - **Select element with indices i and j:** `z <- x[i,j]`
  - **Select i-th element in a matric:** `z <- x[i]` (counted column-wise)
  - **Select i-th row or column:** `z <- x[i,]` or `z <- x[,i]`
  - **Remove i-th row or column:** `z <- x[-i,]` or `z <- x[,-i]`
  - **Select multiple rows or columns:** `z <- x[c(i,j),]` or `z <- x[,c(i,j)]`
  - **Name matrix rows a,b,c:** `rownames(m) <- c('a','b','b')`
  - **Name matrix columns A,B,C:** `colnames(m) <- c('A','B','C')`
  - **Reference a named row or column:** `z <- x['a',]` or `z <- x[,'A']`

Matrix operations

- **Mathematical operations:** (Note: x and y will now be matrices)
  - **Element-wise addition:** `z <- x + y`
  - **Element-wise subtraction:** `z <- x - y`
  - **Element-wise product:** `z <- x * y`
  - **Element-wise division:** `z <- x / y`
  - **Inner product:** `z <- x %*% y`
  - **Outer product:** `z <- x %o% y`

  Note: Most mathematical operators and functions such `^, log, exp`, etc. can be applied to matrices in an element-wise way

  - **Cross product:** `z <- crossprod(x,y)` (equivalent to **x'y**)
  - **Cross product of single matrix:** `z <- crossprod(x)` (equivalent to **x'x**)
  - **Transpose:** `z <- t(x)`
  - **Inverted matrix:** `z <- solve(x)`
  - **Solve equation b=Az for z:** `z <- solve(A,b)` (**A** is matrix, **b** vector)
  - **Generalized inverse:** `z <- ginv(x)` (requires package MASS)
  - **Eigenvalues and vectors:** `z <- eigen(x)`
  - **Singular value decomposition:** `z <- svd(x)`
  - **Cholesky decomposition:** `z <- chol(x)`
  - **QR decomposition:** `z <- qr(x)`
  - **Sums of rows of columns:** `z <- rowSums(x)` or `z <- colSums(x)`
  - **Means of rows and columns:** `z <- rowMeans(x)` or `z <- colMeans(x)`
  - **Apply function FUN to all rows:** `z <- apply(x, 1, FUN)`
  - **Apply function FUN to all columns:** `z <- apply(x, 2, FUN)`

Example: Linear regression function

❖ **Task:** Create your own OLS linear regression function using matrix calculations

Note:

Linear regression model uses the equation:

- $y_i = b_0 + b_1 x_{i,1} + \cdots + b_k x_{i,k} + \varepsilon_i$

Which can be writen in vector form as:

- $y_i = \boldsymbol{b}' \boldsymbol{x}_i + \varepsilon_i$

Where $\boldsymbol{b} = [b_0, \dots, b_k]$, $\boldsymbol{x}_i = [1, x_{i,k}, \dots, x_{i,k}]$

Or in a matrix form for the whole dataset:

- $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{b} + \boldsymbol{\varepsilon}$

Where:

- $\boldsymbol{y} = [y_1, \dots, y_N]$ (N x 1 vector of y)
- $\boldsymbol{X} = [\boldsymbol{x}_1', \dots, \boldsymbol{x}_N']$ (N x k matrix of x)
- $\boldsymbol{\varepsilon} = [\varepsilon_1, \dots, \varepsilon_N]$ (N x 1 vector of ε)
- $\varepsilon_i \sim N(0, \sigma)$ *i.i.d.* for $i = 1, \dots, N$

In the linear-Gassuian case, we can then compute the **OLS estimate** of $\boldsymbol{y}$ with:

- $\widehat{\boldsymbol{b}} = (\boldsymbol{X}'\boldsymbol{X})^{-1}\boldsymbol{X}'\boldsymbol{y}$

Simulation of the dataset:

To test the model, we will simulate our own dataset with $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{b} + \boldsymbol{\varepsilon}$, with the simplifying assumption that $x_i \sim N(0,1)$ for all i

Example: Linear regression function

- **Task:** Create your own OLS linear regression function using matrix calculations
- **Simulate the dataset:**

```
# True parameters
b <- c(1.2,0.9,-0.7) # Parameter vector
sigma <- 0.3 # Error variance
# Number of parameters  (1st is for constant)
k <- length(b)
# Simulate the dataset (X and y)
N <- 10000                    # Observations
X <- matrix(rnorm((k-1)*N),nrow=N,ncol=(k-1))
X <- cbind(rep(1,N),X)        # Add constant
e <- rnorm(N)*sigma           # Random noise
y <- X %*% b + e              # Compute output
```

- **Create linear regression function:**

```
# Linear regression function
mylinreg = function(X,y) {
  be <- solve(t(X) %*% X) %*% t(X) %*% y
  return(be)
}
```

- **Apply the function:**

```
be <- mylinreg(X,y)
print(as.vector(be))
[1] 1.2017280 0.8948191 -0.7024510
```

Note:

Linear regression model uses the equation:

- $y_i = b_0 + b_1 x_{i,1} + \cdots + b_k x_{i,k} + \varepsilon_i$

Which can be writen in vector form as:

- $y_i = \boldsymbol{b}' \boldsymbol{x}_i + \varepsilon_i$

Where $\boldsymbol{b} = [b_0, \dots, b_k]$, $\boldsymbol{x}_i = [1, x_{i,k}, \dots, x_{i,k}]$

Or in a matrix form for the whole dataset:

- $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{b} + \boldsymbol{\varepsilon}$

Where:

- $\boldsymbol{y} = [y_1, \dots, y_N]$ (N x 1 vector of y)
- $\boldsymbol{X} = [\boldsymbol{x}_1', \dots, \boldsymbol{x}_N']$ (N x k matrix of x)
- $\boldsymbol{\varepsilon} = [\varepsilon_1, \dots, \varepsilon_N]$ (N x 1 vector of ε)
- $\varepsilon_i \sim N(0, \sigma)$ *i.i.d.* for $i = 1, \dots, N$

In the linear-Gassuian case, we can then compute the **OLS estimate** of $\boldsymbol{y}$ with:

- $\widehat{\boldsymbol{b}} = (\boldsymbol{X}'\boldsymbol{X})^{-1}\boldsymbol{X}'\boldsymbol{y}$

Simulation of the dataset:

To test the model, we will simulate our own dataset with $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{b} + \boldsymbol{\varepsilon}$, with the simplifying assumption that $x_i \sim N(0,1)$ for all i

Linear regression function

- **Accuracy comparison:**

```
# Compare the true parameters and the fit
comp <- cbind(b,be,b-be)
colnames(comp) <- c('b','be','err')
print(comp)
```

Result:

```
        b      be     err
[1,]  1.2  1.2017 -0.0017
[2,]  0.9  0.8948  0.0052
[3,] -0.7 -0.7025  0.0025
```

- **Alternativelly, we can also write the function as inline function:**

```
# Alternative using inline function
mylinreg2 = function(X,y) solve(t(X) %*% X) %*% t(X) %*% y
```

- **Giving the same result:**

```
be <- mylinreg2(X,y)
print(as.vector(be))
[1] 1.2017280 0.8948191 -0.7024510
```

- **Or by using the R lm function:**

```
# Linear regression with R lm function
fit <- lm(y ~ 0 + X)
fit#coefficients
```

Result:

```
Coefficients:
     X1       X2       X3
 1.2017   0.8948  -0.7025
```

- **Take note:** While our **mylinreg** function estimates the linear regression without constant (which is why we had to add it to the matrix X in advance), the R function **lm** adds the constant automatically. This is why we had to specify the model as **0 + X**, telling R to exclude the constant (as the matrix X already contains it). Alternative would be to use:

`fit <- lm(y ~ X[,-1])` or possibly: `fit <- lm(y ~ 1 + X[,-1])`

Introduction to R

- ✓ Computational software
- ✓ Installation of R and RStudio
- ✓ RStudio layout
- ✓ Data types and basic operations
- ✓ Vectors and sequences
- ✓ Conditions and cycles
- ✓ User build functions
- ✓ Matrices and matrix operations
- ➢ **Lists and data frames**
- ❖ Loading data into R
- ❖ Charts and figures

Lists – Introduction

- **Lists:**
  - Can contain as elements any kind of data type or structure → Including other lists
  - Elements of the list do not need to be of the same type
- **List functions:**
  - **Create a list:** `w <- list(x,y,z)` (x, y and z can be of any type)
  - **Select cell i:** `w[i]`
  - **Select element i:** `w[[i]]`
  - **Named list:** `w <- list('n'=x,'y'=y,'z'=z)`
  - **Select named cell x:** `w['x']` which is equivalent to: `w[i]`
  - **Select element x:** `w$x` which is equivalent to: `w[[i]]`
  - **Variables names:** `names(w)`
  - **Rename variables:** `names(w) <- c('x','y','z')`
  - **Join two lists:** `z <- c(x,y)`
  - **List of lists:** `w <- list(list(a,b,c),list(x,y,z))`

Lists – Example

- **Task:**
  - Create a list that contains a number, integer, complex number, NULL variable, string, logical value, vector, and a matrix

- **Solution:**

```
# Create a list of elements of different types:
x <- list(5, 3L, 2+1i, NULL, 'Hello', TRUE, c(3,6,9), matrix(1:9,3,3))
```

- **Select parts of a list:**

```
# Select part of a list
y <- x[1]

print(y)
[[1]]
[1] 5

typeof(y)
[1] "list"

class(y)
[1] "list"
```

Select element of a list:

```
# Select element of a list
z <- x[[1]]

print(z)
[1] 5

typeof(z)
[1] "double"

class(z)
[1] "numeric"
```

Dataframes – Introduction

- **Dataframes:**
  - Used to store tabular data (such as datasets with different variables)
  - Can contain elements of any data type
  - Elements in each of the columns need to be of the same data type
  - Elements in each column must be of the same type
- **Data frame functions:**
  - **Create a data frame:** `w <- data.frame(x,y,z)`
  - **Select column i:** `w[i]` (output is data frame)
  - **Select vector i:** `w[[i]]` or `w[,i]` (output is vector)
  - **Select row i:** `w[i,]` (output is data frame)
  - **Select column x:** `w['x']` which is equivalent to: `w[i]`
  - **Select vector x:** `w$x` which is equivalent to: `w[[i]]` and `w[,i]`
  - **Variables names:** `names(w)`
  - **Rename variables:** `names(w) <- c('x','y','z')`
  - **Rowwise join:** `z <- rbind(x,y)`
  - **Columnwise join:** `z <- cbind(x,y)`